FINAL PDF TEMPLATE – Arduino Group Project (All Tasks)

Module: DG1FDS – Fundamentals of Digital Systems

Assessment: Arduino Group Project

Group Number: 26

Team Members:

Mohammed Qasem
Huyen Khanh Nung
Victor Emerem
Japheth Mayeye
Josheph Hughes

1. Introduction

1.1 Purpose of the Project

This is a **3-task** embedded systems project using the Grove Beginner Kit for Arduino. The goal is to demonstrate digital systems design, sensor interfacing, and real-time control.

Task 1 Goal: To design and implement a mini weather monitoring system that measures temperature, humidity, and air pressure. Display the sensor readings on the OLED screen and trigger an audible alarm when the temperature exceeds a predefined threshold.

Task 2 Goal: Develop an automatic night-light system that turns on an LED when ambient light levels fall below a set threshold.

Task 3 Goal: Simulate a pedestrian crossing control system. When pedestrians must wait, the RED LED should be on and the OLED should display "STOP". When the pedestrian presses the button:

1.2 Overview of Tasks

Task 1: Mini Weather Station
Task 2: Smart Night-Light System
Task 3: Mini Pedestrian Crossing System

2. Task 1 – Mini Weather Station

2.1 Task Description

Task 1 required the design and implementation of a functional mini weather station capable of monitoring three key environmental measurements: temperature, humidity, and air pressure. Using the Grove Beginner Kit's onboard DHT20 and BMP280 sensors, the system continuously captures real-time atmospheric data and presents the readings on an OLED display for clear and immediate user interpretation.

An important behavioural requirement of this task was the integration of a safety or alert mechanism, achieved through a buzzer that activates whenever the temperature exceeds a predefined threshold. This threshold is adjustable by the user, enabling the system to function in a variety of environmental conditions and making the weather station more flexible and user-responsive. Overall, the task demonstrates core embedded systems principles such as sensor integration, real-time monitoring, user interaction, and conditional output control.

2.2 Components Used

The Mini Weather Station was built entirely using components from the Grove Beginner Kit for Arduino, which provides pre-wired modular sensors and simplifies hardware integration. The DHT20 sensor was used to measure temperature and humidity, providing accurate digital readings via the I2C interface. For atmospheric pressure measurement, the BMP280 sensor was employed, offering high-resolution barometric data suitable for environmental monitoring.

The OLED display (SSD1306-based, I2C) served as the primary visual output, showing real-time sensor values, threshold information, and icon-based feedback. A buzzer connected to digital pin D5 provided audible alerts when temperature exceeded the defined threshold. The potentiometer on analog pin A0 allowed users to adjust the threshold dynamically, improving the system's usability. All components were driven by the Grove Kit's onboard microcontroller, which is Arduino Uno compatible, ensuring seamless programming and hardware compatibility.

2.3 System Logic & Operation

The system operates by continuously acquiring sensor data from the DHT20 and BMP280 modules, which communicate over I2C to provide temperature, humidity, and pressure values. These readings are processed and displayed on the OLED screen using the memory-efficient U8g2 page-buffer mode, which is essential for running graphics on the limited RAM available in

Uno-class boards. The potentiometer plays a key role in user interaction, mapping its analog input to a temperature threshold that determines when the alarm should activate.

If the current temperature rises above this threshold, a non-blocking buzzer routine is triggered, ensuring the buzzer beeps intermittently without disrupting the display refresh or sensor updates. The user interface also includes logic for selecting the appropriate weather icon based on current humidity and temperature conditions, providing a clear visual interpretation of the environment. The entire system loops continuously, ensuring smooth, real-time weather monitoring.

2.4 Extensions Implemented

Temperature bar graph
Dynamic weather icons
Potentiometer-controlled threshold
Memory-optimised OLED rendering

2.5 Final Arduino Code

```
// ===== Mini Weather Station (Grove Beginner Kit) =====
// TEAM GROUP: 26
//
// This program measures temperature, humidity, and air pressure using the
// Grove Beginner Kit. It displays live values on an OLED screen and triggers
// a buzzer alarm if the temperature rises above a user-adjustable threshold.
//
// EXTENSIONS INCLUDED:
// - Temperature bar graph
// - Dynamic weather icons (sun / cloud / droplet)
// - Potentiometer-controlled temperature alert threshold
// - Memory-efficient OLED output using U8g2 page mode

/* ---------------------------------------------------------------------------
   LIBRARIES USED
--------------------------------------------------------------------------- */
#include <Wire.h>
#include "DHT20.h"
#include "Seeed_BMP280.h"
#include <U8g2lib.h>

/* ---------------------------------------------------------------------------
   PIN DEFINITIONS
--------------------------------------------------------------------------- */
#define BUZZER_PIN 5
```

```
#define POT_PIN    A0

/* ---------------------------------------------------------------------------
   SENSOR OBJECTS
--------------------------------------------------------------------------- */
DHT20 dht20;      // Correct object for Grove DHT20
BMP280 bmp280;

/* ---------------------------------------------------------------------------
   OLED DISPLAY SETUP
--------------------------------------------------------------------------- */
U8G2_SSD1306_128X64_NONAME_1_HW_I2C u8g2(
  U8G2_R2, U8X8_PIN_NONE
);

/* ---------------------------------------------------------------------------
   VARIABLES
--------------------------------------------------------------------------- */
float tempC = NAN, humidity = NAN, pressure_hPa = NAN;
float tempThresholdC = 30.0;

const float THRESH_MIN = 15.0;
const float THRESH_MAX = 30.0;

const float TEMP_MIN = 0.0;
const float TEMP_MAX = 30.0;

const unsigned long BEEP_PERIOD_MS = 1000;
const unsigned long BEEP_ON_MS     = 200;
unsigned long t0 = 0;

char lineBuf[24];
char numBuf[10];

/* ---------------------------------------------------------------------------
   SETUP
--------------------------------------------------------------------------- */
void setup() {
  Serial.begin(115200);

  u8g2.begin();

  dht20.begin();        // Correct start method

  if (!bmp280.init()) {
```

```
    Serial.println("BMP280 init failed.");
  }

  pinMode(BUZZER_PIN, OUTPUT);

  u8g2.firstPage();
  do {
    u8g2.setFont(u8g2_font_6x12_tf);
    u8g2.drawStr(0, 14, "Grove Mini Weather");
    u8g2.drawStr(0, 30, "DHT20 + BMP280 + OLED");
    u8g2.drawStr(0, 46, "Pot sets alarm thresh");
  } while (u8g2.nextPage());

  delay(800);
}

/* ---------------------------------------------------------------------------
   LOOP
--------------------------------------------------------------------------- */
void loop() {

  readSensors();

  int potValue = analogRead(POT_PIN);
  tempThresholdC = mapFloat(potValue, 0, 1023, THRESH_MIN, THRESH_MAX);

  updateAlarm();

  drawUI();

  delay(100);
}

/* ---------------------------------------------------------------------------
   SENSOR READING
--------------------------------------------------------------------------- */
void readSensors() {

  dht20.read();                    // Correct method for DHT20

  tempC = dht20.getTemperature();  // access correct values
  humidity = dht20.getHumidity();

  float pPa = bmp280.getPressure();
  pressure_hPa = pPa / 100.0;
```

```
  if (isnan(tempC) || isnan(humidity)) {
    Serial.println("DHT20 read failed.");
  }
}

/* ---------------------------------------------------------------------------
   BUZZER LOGIC
--------------------------------------------------------------------------- */
void updateAlarm() {

  unsigned long now = millis();
  bool alarmActive = (!isnan(tempC) && tempC > tempThresholdC);

  if (alarmActive) {

    unsigned long phase = (now - t0) % BEEP_PERIOD_MS;

    if (phase < BEEP_ON_MS) {
      tone(BUZZER_PIN, 1000);
    } else {
      noTone(BUZZER_PIN);
    }

  } else {
    noTone(BUZZER_PIN);
    t0 = now;
  }
}

/* ---------------------------------------------------------------------------
   OLED UI
--------------------------------------------------------------------------- */
void drawUI() {

  u8g2.firstPage();
  do {

    u8g2.setFont(u8g2_font_6x12_tf);

    u8g2.drawStr(0, 10, "Mini Weather Station");

    formatFloat(tempC, 1, numBuf, sizeof(numBuf), "--.-");
    snprintf(lineBuf, sizeof(lineBuf), "Temp: %s C", numBuf);
    u8g2.drawStr(0, 24, lineBuf);
```

```
    formatFloat(humidity, 1, numBuf, sizeof(numBuf), "--.-");
    snprintf(lineBuf, sizeof(lineBuf), "Humidity: %s %%", numBuf);
    u8g2.drawStr(0, 36, lineBuf);

    formatFloat(pressure_hPa, 1, numBuf, sizeof(numBuf), "----");
    snprintf(lineBuf, sizeof(lineBuf), "Pressure: %s hPa", numBuf);
    u8g2.drawStr(0, 48, lineBuf);

    formatFloat(tempThresholdC, 1, numBuf, sizeof(numBuf), "--.-");
    snprintf(lineBuf, sizeof(lineBuf), "Thresh: %s C", numBuf);
    u8g2.drawStr(0, 60, lineBuf);

    if (!isnan(humidity) && humidity >= 70.0) {
      drawDropletIcon(108, 0);
    } else if (!isnan(tempC) && tempC >= tempThresholdC) {
      drawSunIcon(108, 0);
    } else {
      drawCloudIcon(108, 2);
    }

    drawTempBar(0, 52, 128, 12);

  } while (u8g2.nextPage());
}

/* ---------------------------------------------------------------------------
   BAR GRAPH
--------------------------------------------------------------------------- */
void drawTempBar(int x, int y, int w, int h) {

  u8g2.drawFrame(x, y, w, h);

  float t = tempC;
  if (isnan(t)) t = TEMP_MIN;
  t = constrain(t, TEMP_MIN, TEMP_MAX);

  int fillW = (int)((t - TEMP_MIN) * (w - 2) / (TEMP_MAX - TEMP_MIN));

  u8g2.drawBox(x + 1, y + 1, fillW, h - 2);

  for (int tick = 10; tick <= 40; tick += 10) {
    int tx = x + 1 + (tick - TEMP_MIN) * (w - 2) / (TEMP_MAX - TEMP_MIN);
    u8g2.drawVLine(tx, y, 3);
  }
```

```
}

/* ---------------------------------------------------------------------------
   ICONS
--------------------------------------------------------------------------- */
void drawSunIcon(int x, int y) {
  int cx = x + 10, cy = y + 10, r = 6;
  u8g2.drawCircle(cx, cy, r, U8G2_DRAW_ALL);
  u8g2.drawLine(cx - 10, cy, cx - 6, cy);
  u8g2.drawLine(cx + 6, cy, cx + 10, cy);
  u8g2.drawLine(cx, cy - 10, cx, cy - 6);
  u8g2.drawLine(cx, cy + 6, cx, cy + 10);
}

void drawCloudIcon(int x, int y) {
  u8g2.drawCircle(x + 6,  y + 8, 6, U8G2_DRAW_ALL);
  u8g2.drawCircle(x + 14, y + 6, 8, U8G2_DRAW_ALL);
  u8g2.drawCircle(x + 22, y + 9, 5, U8G2_DRAW_ALL);
  for (int i = 8; i <= 14; ++i) {
    u8g2.drawHLine(x + 2, y + i, 24);
  }
}

void drawDropletIcon(int x, int y) {
  int cx = x + 10, cy = y + 12;
  u8g2.drawLine(cx, y, cx - 6, cy - 4);
  u8g2.drawLine(cx, y, cx + 6, cy - 4);
}

/* ---------------------------------------------------------------------------
   HELPERS
--------------------------------------------------------------------------- */
void formatFloat(float v, uint8_t decimals, char* out, size_t outLen, const
char* fallback) {
  if (isnan(v)) {
    strncpy(out, fallback, outLen);
    out[outLen - 1] = '\0';
  } else {
    dtostrf(v, 0, decimals, out);
  }
}

float mapFloat(long x, long in_min, long in_max, float out_min, float out_max)
{
  return (float)(x - in_min) * (out_max - out_min) / (float)(in_max - in_min)
+ out_min;
}
```

3. Task 2 – Smart Night-Light System

3.1 Task Description

Task 2 required the development of an automatic Smart Night-Light system that responds to changes in ambient light levels. The goal of the task was to create a system that turns on an LED when the surrounding environment becomes dark, simulating how real-world night-lights or pathway lights operate. The onboard light sensor (LDR) of the Grove Beginner Kit provides continuous readings that reflect the brightness of the environment, and the system compares these values against a user-defined threshold set using a potentiometer.

When the light level drops below this adjustable threshold, the LED turns on, and when the light level rises above it, the LED switches off. This demonstrates practical applications of analogue sensing, threshold detection, and actuator control, while also reinforcing key embedded-systems concepts such as environmental monitoring, real-time decision-making, and user-adjustable system behaviour.

3.2 Components Used

The Smart Night-Light System makes use of several onboard components from the Grove Beginner Kit, eliminating the need for external wiring. The primary sensor is the Light-Dependent Resistor (LDR), which measures ambient light levels by varying its resistance in response to brightness. An LED serves as the system's output indicator and turns on whenever the environment becomes dark based on the threshold set by the user.

The potentiometer on analog pin A0 allows the user to adjust this threshold dynamically during operation, providing a convenient interface for tuning sensitivity. An optional addition to the system is the OLED display, which can be used to present live light-level data, threshold values, or visual indicators showing when the LED is active. All components are controlled by the Grove Kit's central microcontroller, offering a simple yet effective platform for implementing this intelligent lighting system.

3.3 System Logic

The operation of the Smart Night-Light System is based on continuous monitoring of analogue light levels and comparing them to a user-defined threshold. The LDR outputs a voltage that varies depending on brightness: higher values under bright light and lower values in darker

environments. The potentiometer provides another analogue value, which is mapped to define the threshold at which the LED should turn on.

The microcontroller repeatedly checks whether the ambient light is below this threshold; if it is, the LED is activated, and if not, the LED remains off. This decision-making process occurs in real time within the main loop, ensuring quick and smooth responses to changes in lighting conditions. If the OLED display is used, it can show the live sensor readings as well as a bar graph or icon, helping the user visually understand how close the current light level is to triggering the LED. The logic is simple, efficient, and highlights fundamental embedded-system design principles such as continuous sampling, threshold comparison, and output actuation.

3.4 Extensions

PWM brightness control
OLED live light-level graph
Icons (sun/moon)

3.5 Final Code

```
/*
  Smart Night-Light System ( Grove Beginner Kit)
  Team Group : 26

  Task Description:
   this is an automatic night-light system that can turn on a
  LED when ambient light falls below a set threshold.

  Components :
   light Sensor (LDR)- A6
   Potentiometer - A0 (threshold adjustment)
   LED - D4
   OLED Display-I2C (SSD1306,128*64)

  Extension (optional):
   PWM brightness control ( LED brightness varies with light level).
   OLED display showing real- time values and bar graph.
   Threshold marker : visual indicator on bar graph
   Dynamic Icons : Sun ( day)/ Moon ( night)
*/

//===== Libraries =====
#include <Wire.h>     //I2C communication for OLED
#include <U8g2lib.h>    // OLED display library
```

```
//==== Pin =====

const int ledPin = 4;  // select the pin for the LED
const int light_sensor = A6;  // Light sensor

int light_Level = 0;   // Light value from sensor

const int potPin = A0; // potentiometer (threshold adjust)
int threshold = 0;

int ledBrightness = 0;  // PWM brightness value (0-255)
bool isDark= false;    // true when lightlevel < threshold

//==== OLED DISPLAY SETUP ======

// Using PAGE BUFFER MODE (_1_) to save precious RAM on Arduino Uno
// U8G2_R2 rotates screen 180° for correct viewing orientation on Grove Kit

U8G2_SSD1306_128X64_NONAME_1_HW_I2C u8g2(
  U8G2_R2,  // rote 180 degrees
  /* reset = */ U8X8_PIN_NONE // no reset pin needed
);

//==== display scaling ====
const int light_min = 0; // minimum light value
const int light_max = 1023 ; // maximum light value
const int threshold_min = 0; // minimum threshold value
const int threshold_max = 1023; // maximum threshold value

// Display buffer (reusable to save memory)
char lineBuffer[30];                    // For formatting text lines
char numberBuffer[10];                  // For formatting numbers

//===== funtion ====
void setup() {
  // declare the ledPin as an OUTPUT
  pinMode(ledPin, OUTPUT);
  analogWrite (ledPin,0);
  Serial.begin(9600);
  // initializes the serial port with a baud rate of 9600

  u8g2.begin();  // initializes OLED display
  displaySplashScreen();
  delay (2000);
```

```
}

void loop() {

  //  Read all sensors
  readSensors();

  // Process light control logic (with PWM)
  processLightControl();

  //Update OLED display
  updateDisplay();

  //Print debug information to Serial Monitor
  printDebugInfo();

  delay(200);  // delay in between reads for stability
}

void readSensors() {
  light_Level = analogRead(light_sensor);
  // read light levl from LDR sensor ( 0 = dark , 1023 = bright)
  threshold = analogRead(potPin);
  // read threshold value from potentiomenter
}

void processLightControl() {

  if (light_Level < threshold) {

    isDark = true;

    // Calculate brightness based on darkness level
    int darknessLevel = threshold - light_Level;
    // larger = darker
    ledBrightness = map(darknessLevel, 0, threshold, 0, 255);
    // this code  makes LED brighter if it's gets darker
    ledBrightness = constrain(ledBrightness, 0, 255);
    // this code ensure brightness stays within valid range (0-255)

  } else {
      isDark = false;
      ledBrightness = 0;
      // turn LED off - bright condition
```

```
  }

  analogWrite(ledPin, ledBrightness);
}

void updateDisplay() {
  u8g2.firstPage();
  do {
    u8g2.setFont(u8g2_font_6x12_tf);
    // set font for all text

    // ----- Title Line -----
    u8g2.drawStr(0, 10, "Smart Night Light");

    // ----- Light Level Display -----
    formatNumber(light_Level, numberBuffer, sizeof(numberBuffer), "---");
    snprintf(lineBuffer, sizeof(lineBuffer), "Light: %s", numberBuffer);
    u8g2.drawStr(0, 24, lineBuffer);

    // ----- Threshold Display -----
    formatNumber(threshold, numberBuffer, sizeof(numberBuffer), "---");
    snprintf(lineBuffer, sizeof(lineBuffer), "Thres: %s", numberBuffer);
    u8g2.drawStr(0, 36, lineBuffer);

    // ----- LED Status & Brightness -----
    if (isDark) {
      formatNumber(ledBrightness, numberBuffer, sizeof(numberBuffer), "---");
      snprintf(lineBuffer, sizeof(lineBuffer), "LED: ON (%s)", numberBuffer);
    } else {
      snprintf(lineBuffer, sizeof(lineBuffer), "LED: OFF");
    }
    u8g2.drawStr(0, 48, lineBuffer);


    // ----- Status Icon (Top-right corner) -----
    // Sun icon when bright, Moon icon when dark
    if (isDark) {
      drawMoonIcon(100, 5);  // Moon icon for night mode
    } else {
      drawSunIcon(108, 5);    // Sun icon for day mode
    }

    // ===== BAR GRAPH VISUALIZATION (Optional Extension) =====
    // Draw bar graph showing light level relative to min/max
    drawLightBarGraph(0, 52, 128, 12);
```

```
    // Draw threshold marker on the bar graph
    drawThresholdMarker(0, 52, 128, 12);

  } while (u8g2.nextPage());
}

void drawLightBarGraph(int x, int y, int w, int h) {
  // Draw frame
  u8g2.drawFrame(x, y, w, h);

  // light level
  int displayLight = constrain(light_Level, light_min, light_max);

  // Calculate fill width
  int fillWidth = map(displayLight,light_min, light_max, 0, w - 2);

  // Draw bar chart
  if (fillWidth > 0) {
    u8g2.drawBox(x + 1, y + 1, fillWidth, h - 2);
  }
}

// Draw threshold marker on the bar graph
void drawThresholdMarker(int x, int y, int w, int h) {
  // Map threshold value
  int markerX = map(threshold, threshold_min, threshold_max, x, x + w - 1);

  // Draw vertical line as marker
  u8g2.drawLine(markerX, y - 3, markerX, y + h + 2);

}

// ============================ ICON DRAWING FUNCTIONS
============================

// Draw sun icon (for bright conditions)
void drawSunIcon(int x, int y) {

  int cx = x + 10, cy = y + 10;

  // center of the Sun
  u8g2.drawDisc(cx, cy, 6, U8G2_DRAW_ALL);

  // Circle Face
  u8g2.setDrawColor(0);  // 0 = black
```

```
  // Draw black eye
  u8g2.drawDisc(cx - 3, cy - 3, 1, U8G2_DRAW_ALL);  // left eye
  u8g2.drawDisc(cx + 3, cy - 3, 1, U8G2_DRAW_ALL);  // right eye

  // draw smiling mouth (black)
    u8g2.drawLine(cx - 4, cy + 3, cx, cy + 5);
  u8g2.drawLine(cx, cy + 5, cx + 4, cy + 3);

  // change colour to white for sun rays
  u8g2.setDrawColor(1);  // 1 = white

  // Draw 4 main sun rays (top, bottom, left, right)
  u8g2.drawLine(cx, cy - 10, cx, cy - 7);  // Top ray
  u8g2.drawLine(cx, cy + 7, cx, cy + 10);  // Bottom ray
  u8g2.drawLine(cx - 10, cy, cx - 7, cy);  // left ray
  u8g2.drawLine(cx + 7, cy, cx + 10, cy);  // Right ray

  // Draw 4 rays
  u8g2.drawLine(cx - 7, cy - 7, cx - 5, cy - 5);  // top left
  u8g2.drawLine(cx + 5, cy + 5, cx + 7, cy + 7);  // bottom right
  u8g2.drawLine(cx - 7, cy + 7, cx - 5, cy + 5);  // bottom left
  u8g2.drawLine(cx + 5, cy - 5, cx + 7, cy - 7);  // top right
}
// Draw moon icon (for dark conditions)
void drawMoonIcon(int x, int y) {

  // draw full moon
  u8g2.drawDisc(x + 10, y + 10, 7, U8G2_DRAW_ALL);

  u8g2.setDrawColor(0);   // Set color to black
  u8g2.drawDisc(x + 14, y + 8, 5, U8G2_DRAW_ALL);
  u8g2.setDrawColor(1);   // Restore to white
}

// ===== Helper =====

// Format numbers for display (handles invalid values)
void formatNumber(int value, char* buffer, size_t bufferSize, const char*
invalidText) {
  if (value >= 0 && value <= 1023) {
    snprintf(buffer, bufferSize, "%d", value);
  } else {
    snprintf(buffer, bufferSize, "%s", invalidText);
  }
}
```

```
// Print debug information to Serial Monitor
void printDebugInfo() {
  Serial.print(F("Light: "));
  Serial.print(light_Level);
  Serial.print(F(" | Threshold: "));
  Serial.print(threshold);
  Serial.print(F(" | LED: "));
  if (isDark) {
    Serial.print(F("ON (PWM="));
    Serial.print(ledBrightness);
    Serial.println(F(")"));
  } else {
    Serial.println(F("OFF"));
  }
}

// Display splash screen on startup
void displaySplashScreen() {
  u8g2.firstPage();
  do {
    u8g2.setFont(u8g2_font_6x12_tf);
    u8g2.drawStr(20, 20, "Smart Night");
    u8g2.drawStr(25, 35, "Light System");
    u8g2.drawStr(10, 50, "Task 2 - Group 26");
  } while (u8g2.nextPage());
}

// Thank You//
```

4. Task 3 – Mini Pedestrian Crossing System

4.1 Task Description (from brief)

Task 3 required the design and implementation of a simplified pedestrian crossing control system, similar to those found at real-world traffic intersections. The system begins in a default STOP state, indicated by a red LED and a “STOP” message on the OLED display. When the pedestrian push button is pressed, the system transitions into a WAIT state, displaying a countdown timer on the OLED to inform the user how long they need to wait before it is safe to cross.

Once the countdown reaches zero, the system enters the GO state, during which the buzzer provides audible feedback and the OLED displays a “GO” or crossing symbol to signal that pedestrians may safely move.

After the crossing time expires, the system automatically returns to the STOP state. This task reinforces key embedded-system concepts such as state-machine logic, input handling (button press), timed sequences, and synchronised multimodal output using LEDs, a buzzer, and an OLED display.

4.2 Components Used

The Mini Pedestrian Crossing System uses several components from the Grove Beginner Kit to simulate a realistic crossing interface. A momentary push button serves as the pedestrian request input, allowing the user to initiate the crossing sequence. A red LED acts as the STOP indicator, clearly signalling when crossing is not permitted. The onboard buzzer provides audible cues during the GO phase, enhancing accessibility and mimicking the real-world audible crossing alerts used to assist visually impaired pedestrians.

The OLED display is used for user-friendly visual communication, including showing “STOP”, “WAIT”, the countdown timer, and “GO”. Optional extensions such as flashing LEDs, animated countdown icons, or pre-warning buzzer tones can be added to improve realism and usability. All components are controlled by the microcontroller built into the Grove Beginner Kit, allowing reliable management of timing, state changes, and user input.

4.3 State Machine Explanation

The Mini Pedestrian Crossing System operates using a simple but robust three-state finite-state machine (FSM). **State 1: STOP** is the default condition where the red LED remains on and the OLED displays “STOP”. The system waits in this state until the push button is pressed. A valid button press transitions the system into **State 2: WAIT**, where a countdown timer is displayed to indicate how long the user must wait before crossing; during this time, the system ensures no GO signals are shown. Once the countdown completes, the system transitions into **State 3: GO**, during which the buzzer generates audible feedback and the OLED displays “GO” to indicate safe crossing.

After a predetermined GO duration expires, the system automatically reverts to the STOP state and begins monitoring for new button presses. This structured FSM approach ensures predictable, safe, and realistic behaviour that closely models an actual pedestrian crossing system.

4.4 Extensions

Flashing LED
Countdown numbers
Return-to-STOP warning

4.5 Final Arduino Code

```
// ===== Mini Pedestrian Crossing System (Grove Beginner Kit) =====
// TEAM GROUP: 26
//
// This program simulates a simple pedestrian crossing system using the
// Grove Beginner Kit. The system starts in the STOP state, indicated by a
// red LED and a "STOP" message on the OLED display. When the push button is
// pressed, the system enters a WAIT countdown shown on the OLED. After the
// wait period ends, the system transitions into the GO state where the buzzer
// provides audible feedback and a "GO" countdown is displayed. After the
// crossing period finishes, the system returns automatically to the STOP
state.
//
// FEATURES INCLUDED:
// - STOP → WAIT → GO state-machine behaviour
// - OLED display feedback for each phase (STOP, WAIT, GO)
// - Countdown timers for both WAIT and GO phases
// - Buzzer alerts for transition and end-of-crossing warnings
// - Red LED indicator for STOP state
//
// ---------------------------------------------------------------------------
//   LIBRARIES USED
//   These libraries allow the Arduino to:
//   - Communicate via I2C using Wire
//   - Display text on the SSD1306 OLED using the SSD1306Ascii library
// ---------------------------------------------------------------------------
// Include the Wire library needed for I2C communication
#include <Wire.h>

// Include libraries for writing text to the SSD1306 OLED display
#include "SSD1306Ascii.h"
#include "SSD1306AsciiWire.h"

// Create an OLED display object (using I2C via the Wire interface)
SSD1306AsciiWire display;

// Grove Beginner Kit pins
const int buttonPin = 6;   // Push button used by pedestrians to request
crossing
```

```
const int ledPin = 4;      // Red LED used to signal STOP
const int buzzerPin = 5;   // Buzzer for beeps during crossing phases

// Timings (in seconds)
int waitTime = 5;          // Duration of WAIT countdown after button is
pressed
int crossTime = 10;        // Duration of GO countdown (time allowed to cross)

void setup() {

  // Configure pins for input/output
  pinMode(buttonPin, INPUT);     // Button as an input
  pinMode(ledPin, OUTPUT);       // LED as an output
  pinMode(buzzerPin, OUTPUT);    // Buzzer as an output

  // Start I2C communication for the OLED display
  Wire.begin();

  // Initialize the OLED display with correct resolution and I2C address
  display.begin(&Adafruit128x64, 0x3C);

  // Choose a readable default font
  display.setFont(System5x7);
}

// Function to display the STOP message
void showStop() {
  display.clear();   // Wipe the screen
  display.set2X();   // Double-size text for visibility
  display.println("STOP");
}

// Function to display countdown before GO phase
void countdown(int time) {

  // Loop from the given time down to 1
  for(int i = time; i > 0; i--) {
    display.clear();          // Clear screen for updated number
    display.set2X();          // Large text
    display.print("WAIT ");   // Display WAIT label
    display.println(i);       // Display countdown number
    delay(1000);              // Wait one second per step
  }
}

// Function to display GO countdown during crossing phase
```

```
void crossing(int time) {

  for(int i = time; i > 0; i--) {
    display.clear();        // Clear display for the new value
    display.set2X();        // Large text for visibility
    display.print("GO ");   // Indicate GO state
    display.println(i);     // Show countdown
    delay(1000);            // 1 second delay
  }
}

void loop() {

  // Default state: STOP state active
  digitalWrite(ledPin, HIGH);  // Turn ON the RED LED (STOP)
  showStop();                  // Display STOP on OLED

  // Check if the pedestrian has pressed the button
  if(digitalRead(buttonPin) == HIGH) {

    delay(200); // Small debounce delay to avoid multiple triggers

    // Show WAIT countdown on OLED
    countdown(waitTime);

    // Give a brief beep to indicate transition to GO
    tone(buzzerPin, 1000);   // Play beep at 1 kHz
    delay(500);              // Beep duration
    noTone(buzzerPin);       // Stop the buzzer

    // Turn OFF the red LED since crossing will start
    digitalWrite(ledPin, LOW);

    // Display GO countdown while pedestrians can cross
    crossing(crossTime);

    // After GO period ends, give three warning beeps before returning to STOP
    for(int i = 0; i < 3; i++){
      tone(buzzerPin, 2000);  // Short high-pitched warning beep
      delay(200);
      noTone(buzzerPin);
      delay(300);             // Pause between beeps
    }
  }

}
```

5. Group Contribution Table

Student Name	Contribution (%)	Description of Contribution
Mohammed Qasem	100%	Coding Task 1, Testing, Documentation, Video editing
Huyen Nung	100%	Task 2 coding, wiring
Victor Emerem	100%	Task 3 logic, OLED graphics
Japheth Mayeye	100%	Task 3 logic, OLED graphics
Josheph Hughes	0%	

6. Conclusion

The Arduino group project successfully demonstrated the design and implementation of three functional embedded systems using the Grove Beginner Kit: a Mini Weather Station, a Smart Night-Light System, and a Mini Pedestrian Crossing System. Across these tasks, the team achieved full integration of multiple sensors, actuators, displays, and user-input mechanisms while ensuring that each system operated reliably and in real time.

Through completing the project, the team developed essential technical skills including digital and analogue sensor interfacing, microcontroller programming, threshold-based decision making, timing control, and state-machine logic development. The use of libraries, memory-efficient display techniques, and non-blocking code further strengthened the team's understanding of embedded-system constraints and optimisation strategies.

Beyond technical skills, the project reflects real-world applications such as environmental monitoring, automated lighting, and pedestrian safety systems technologies commonly used in smart homes, IoT devices, and modern urban infrastructure. Overall, the project enhanced both practical engineering ability and problem-solving competence, preparing the team for more advanced work in embedded systems and digital electronics.